1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L25 – Code Testing**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4

5

Code Testing

6 **Why?**

- THREE reasons

7 **Why?**

1. To know if it is written with proper syntax (so that it compiles) – syntactical correctness

8 **Why?**

1. To know if it is written with proper syntax (so that it compiles) – syntactical correctness
2. To know if it performs what it is meant to perform – logically accurate

9 **Why?**

1. To know if it is written with proper syntax (so that it compiles) – syntactical correctness
2. To know if it performs what is is meant to perform – logically accurate
3. To know if it follows "convention" -so that others can understand and maintain

10 **How to Test**

- Syntax – code compilers (typically inbuilt with IDEs)

11 **How to Test**

- For the other two types
  - More complex
  - Several methods

12 **Testing Methods**

- TWO broad categories of methods
  - Review-based

- Execution-based

13

Code Review

14 **What**

- Relatively less rigorous and quick testing method
  - Carried out after a module is successfully compiled and all syntax errors eliminated

15 **Purpose**

- Cost-effective strategies for reduction in coding errors and produce high quality code

16 **How It Is Done**

- TWO broad approaches

17 **How It Is Done**

- Code inspection (similar to heuristic evaluation)
  - Evaluation team checks code for violation of coding standards

18 **How It Is Done**

- Code walkthrough (similar to cognitive walkthrough)
  - Evaluation team manually executes code for some (representative) test cases to identify logical errors

19 **How It Helps!**

- Quick method – saves development time
- Inspection helps to conform to good coding practices – helps in maintainability and portability of code
- Walkthrough – allows to quickly identify logical errors

20

Execution-Based Testing

21 **Code Testing**

- Review-based testing informal – mostly evaluates code qualitatively
- Good for early evaluation to 'clean up' the code before more rigorous and formal testing is done

22 **(Formal) Program Testing**

- Consists of
  - Providing program a set of test inputs (or test cases)
  - Observing program behavior (and/or output)

23 **(Formal) Program Testing**

- If program fails to behave as expected (or output mismatch), the conditions under which failure occurs are noted for later debugging and correction

24 **Terminology**

- Test case – a triplet [I,S,O]

- I = data input
- S = system state at input time
- O = expected output

- For simplicity, we will consider only the doublet [I,O]

25 **Terminology**

- Test suite - set of all test cases with which a given software is to be tested

26 **Note**

- Aim of testing - to identify ALL defects in a software product

27 **Note**

- In practice, not possible to guarantee software is *completely error free* after testing
  - Input data domain of most software products very large
  - Not practical to test software exhaustively with respect to every possible input

28 **Note**

- Then why to go for testing at all!

29 **Note**

- Testing does expose many (most) defects (important ones) if done properly and systematically
  - Practical way of reducing defects in a system
  - Increases confidence in a developed system

30 **Choice of Test Cases (Suite)**

- Exhaustive testing impractical - possible input data values extremely large or infinite
  - We must design test suite that is of reasonable size and can uncover as many errors existing in the system as possible

31 **Choice of Test Cases (Suite)**

- Randomly selected test cases not necessarily contribute to significance of test suite - they need not detect additional defects not already detected by other test cases
  - Number of test cases not indication of testing effectiveness
  - Large number of test cases selected at random does not guarantee all (or even most) of the errors will be uncovered

32 **Choice of Test Cases (Suite)**

- Example - code to find greater of two integers

```
If (x>y) max = x;
else max = x;
```

(code has a simple programming error)

33 **Choice of Test Cases (Suite)**

```
If (x>y) max = x;
else max = x;
```

- Consider test suite,

  Case 1: (x=3,y=2), 3

  Case 2: (x=2,y=3), 3

- Can detect the error

34 **Choice of Test Cases (Suite)**

If (x>y) max = x;
else max = x;

- Consider a larger test suite
  Case 1: (x=3,y=2), 3
  Case 2: (x=4,y=3), 4
  Case 3: (x=5,y=1), 5
- Can't detect the error ☐ larger test suite not necessarily better always

35 **Choice of Test Cases (Suite)**

- Implication - test suite should be carefully designed (not decided randomly)
- Require systematic approaches

36 **(Systematic) Code Testing**

- Broadly of TWO types
  - Functional testing - test cases designed using only functional specification of software, i.e. without any knowledge of internal structure [Black-box testing]
  - Structural testing - test cases designed using knowledge of internal structure of software [white-box testing]

37 **Note**

- We shall learn in details about all these testing techniques in the subsequent lectures, with suitable case studies

38 **Reference**

- Rajib Mall (2018). Fundamentals of Software Engineering, 5th ed, PHI Learning Pvt Ltd. Chapter 10
- Roger S Pressman (2020). Software Engineering: A Practitioner's Approach, 9th ed, McGraw-Hill Education, New York, Chapter 19-21